PS-8 Study Guide — Group 1

Goal of this guide: every group member should be able to explain *any* part of the project — the idea, the code, the results and their limitations — during the evaluation (11 Nov 2026, 10:00–10:15). The guidelines say: *"Ensure that all group members understand the methodology, code, results, and conclusions and can explain them."*

How to use it:

1. Read **Part 1** (5 minutes) — the whole project in one page.
2. Read **Part 2** for any concept you are unsure about.
3. Learn the **numbers in Part 5** and practise the **questions in Part 7** out loud.
4. On the morning of the evaluation, go through the **checklist in Part 9**.

Part 1 — The project in one page

Problem statement (PS-8): *Evaluate SBERT-based sentence embeddings for predicting story points of later issues in a target project using chronological ordering.*

That sentence contains three choices:

Part of the title	What it means for us
SBERT sentence embeddings	We turn every *sentence* of an issue into a vector of 768 numbers using a pre-trained SBERT model.
Target project (cross-project)	We train on one project (the *source*) and test on a *different* project (the *target*).
Later issues, chronological ordering	Issues are sorted by creation date; we test on the **latest 20 %** of the target project.

What we built (the pipeline):

```
TAWOS dataset → clean the issues → split text into sentences
   → SBERT: each sentence → 768 numbers → each issue = 24 × 768 matrix
   → 3-layer CNN → predicted story points
```

How we evaluated: 39 source → target project pairs (given in the guidelines). For each pair: choose settings with 5-fold validation on the source project, train the final model, test once on the latest 20 % of the target.

Main result (memorise this):

	MAE	MdAE
Our model (SBERT + CNN)	**2.21**	**1.85**
Median baseline (always predict the source project's median)	2.16	1.88
Mean baseline (always predict the source project's mean)	2.56	2.19

The story in three sentences:

1. Our model has the **lowest MdAE**, but its MAE is about the **same as the median baseline** (no significant difference, Wilcoxon p = 0.59).

2. **Inside the project it was trained on, the model beats the median baseline in 34 of 39 pairs** — so the text does contain some information about effort.
3. On a **new** project this advantage disappears, mainly because **every team uses its own story point scale**.

> Do not be embarrassed that we did not beat the baseline. Published research found the same thing (Tawosi et al., 2023: deep-learning estimators are often not better than the median baseline). Reporting it honestly is exactly what the guidelines ask for: *"do not fabricate results or claim improvements unsupported by measurements."*

Part 2 — Key concepts explained simply

2.1 Story points

In agile development, work is split into **issues** (tasks / user stories). Before starting, the team estimates the effort of each issue in **story points** — a *relative* number, often from the Fibonacci-like scale 1, 2, 3, 5, 8, 13. "3 points" has no fixed meaning: each team decides what its numbers mean. **This is the most important fact for understanding our results.**

2.2 Embeddings

A neural network cannot read text, so text is turned into a list of numbers (a **vector**) called an **embedding**.

Type	One vector per …	Example model	Used by
Word embedding	word / token	BERT	PS-1 to PS-4
Sentence embedding	**whole sentence**	**SBERT**	**PS-5 to PS-8 (us)**
Combined	both	BERT + SBERT	PS-9 to PS-12

2.3 BERT vs SBERT

- **BERT** gives a vector for every word, depending on the words around it (contextual).
- **SBERT (Sentence-BERT)** is BERT trained further so that **sentences with similar meaning get similar vectors**. It averages the word vectors ("mean pooling") into one sentence vector.
- We used the pre-trained SBERT model `all-mpnet-base-v2` (from the `sentence-transformers` library). It outputs **768 numbers per sentence** and is one of the most accurate general-purpose models in that library.
- We kept SBERT **frozen** (we did not train it further). We only use it to produce vectors; only the CNN is trained.

2.4 Why sentences, and why a matrix?

An issue has a title and a description with several sentences. We give each sentence its own SBERT vector, in order:

```
row 1  : SBERT(title)                    → 768 numbers
row 2  : SBERT(description sentence 1)   → 768 numbers
row 3  : SBERT(description sentence 2)   → 768 numbers
...
row 24 : zeros (padding, if the issue is shorter)
```

So every issue is a **24 × 768 matrix**. 24 sentences cover **97.1 %** of issues (median issue = 4 sentences). A **mask** (true/false per row) tells the CNN which rows are real and which are padding.

2.5 The CNN (convolutional neural network)

A **1-D CNN** slides small "filters" along the sentence rows.

- **Kernel size 3** = each filter looks at **3 consecutive sentences** at a time.
- **Filters** = pattern detectors. Layer 1 has 256, layer 2 has 128, layer 3 has 64.
- **ReLU** = activation function: keeps positive values, sets negatives to 0 (adds non-linearity).
- **Dropout** = randomly switches off some neurons during training to reduce **overfitting**.
- **Pooling** = turns "one value per sentence position" into one value per filter: we take both the **maximum** and the **average** over the real sentences (64 + 64 = 128 numbers).
- **Fully connected layers** = 128 → 64 → **1 output = predicted story points**.
- About **0.72 million** trainable parameters. *Why three layers?* The guidelines suggest a three-layer CNN.

```
24×768 —Conv(256,k=3)—► 24×256 —Conv(128,k=3)—► 24×128 —Conv(64,k=3)—► 24×64
       —max + mean pool—► 128 —► 64 —► 1 (story points)
```

2.6 Training words you should know

Term	Meaning in our project
Loss function	What training minimises. We used **L1 loss** = average absolute error = **MAE** itself.
Optimiser	Algorithm that updates the weights. We used **AdamW** (Adam + weight decay 0.0001).
Learning rate	Size of each update step. We tried 0.001 and 0.0003.
Epoch	One full pass over the training data. Max 60.
Early stopping	Stop when the validation MAE has not improved for 8 epochs; keep the best weights.
Batch size	64 issues are processed together in one update.
Random seed	Fixes the randomness so results can be reproduced. We used 42, 43, 44.
Overfitting	Model memorises training data and does badly on new data. Dropout + early stopping fight it.

2.7 Splitting the data

Random split vs chronological split: a random split mixes old and new issues, so the model can learn from issues created *after* the ones it is tested on — that is unrealistic ("data leakage"). A **chronological** split trains on the past and tests on the future, like real life.

Within-project vs cross-project: within-project = train and test on the same project. **Cross-project (ours)** = train on project A, test on project B. Harder, but realistic for a *new* project with no history.

Our split for one pair:

```
Target (sorted by date): [ earlier 80 % — NOT USED      ][ latest 20 % = TEST ]
Source (sorted by date): [ Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 ]
                           (training + validation)
```

- **Test set** = latest 20 % of the *target* project (80:20 chronological split). Used **once**, at the very end.
- The earlier 80 % of the target is not used at all → the model learns *nothing* from the target project.

- **5-fold validation** on the *source*: cut into 5 time-ordered folds. Run 5 times; each time 4 folds train and 1 fold validates; every fold is validation once. The average validation MAE chooses the settings.

2.8 Forward-chaining (if asked about "future data in training")

In normal 5-fold validation, when fold 1 is the validation fold, the model trains on folds 2–5, which are *newer*. Someone may ask whether that is a problem. We checked with **forward-chaining**: split into 6 time blocks; run *k* trains on blocks 1…k and validates on block k+1 → validation is *always* newer than training. Result: **1.76** vs **1.82** — almost the same, so it did not matter. (The *test* set is always strictly the latest 20 %.)

2.9 Metrics: MAE and MdAE (with an example)

For each test issue, error = |true − predicted|.

Issue	True SP	Predicted	Absolute error
A	1	2	1
B	3	3	0
C	5	2	3

- **MAE** (mean absolute error) = (1 + 0 + 3) / 3 = **1.33**
- **MdAE** (median absolute error) = median(0, 1, 3) = **1**
- Both are in story points; **lower is better**. MdAE ignores a few very large errors, MAE does not.

2.10 Baselines and why the median baseline is strong

A **baseline** is a very simple predictor that does not read the text. If our model cannot beat it, the text is not helping.

- **Median baseline:** always predict the median story point of the training project.
- **Mean baseline:** always predict the average.

Math fact: the single number that minimises the **sum of absolute errors** is the **median**. So for MAE, the median baseline is a *very* strong opponent.

2.11 Statistics used (only if asked)

- **Wilcoxon signed-rank test:** compares two methods over the 39 pairs (paired). **p < 0.05** = significant difference. Ours vs median baseline: **p = 0.59 → no significant difference.**
- **Correlation r:** −1 to +1; how strongly two things move together. Story-point scale gap vs error: **r = 0.76** (strong).

Part 3 — Dataset and preprocessing

- **TAWOS** (Tawosi et al., MSR 2022): Jira issues of open-source agile projects. The current release has **39 projects from 12 organisations** (Apache, Atlassian, MongoDB, Hyperledger, …). Shared as one MySQL dump: **637 MB zipped, 4.3 GB unzipped**.
- We did **not** install MySQL: our script reads the SQL file straight from the zip and keeps only the `Issue`, `Project` and `Repository` tables (~1.5 minutes).
- All **39 projects** have story points — exactly the 39 projects of the guideline pair list.

Cleaning steps (numbers to know):

Step	Issues left
Issues with a story point	65,427
Only completed work (Fixed / Done / Complete …; not Won't Fix, Duplicate, still open)	53,901
0 < story point ≤ 100 (the raw maximum was **26,710,114** — a data-entry error)	51,992
Remove exact duplicates	**51,777**

Text used: title + description. TAWOS also provides the description *without code snippets* (`Description_Text`), which we used. Links → "URL", user names → "USER", Jira formatting removed, then split into sentences. 305,403 sentences in total, 248,947 unique.

The one assumption we made: the guideline lists "C++ driver (Appcelerator)", which does not exist in TAWOS. The only unmatched TAWOS project is **Aptana Studio**, which belongs to the Appcelerator organisation, so we used it.

Project differences: sizes range from 103 issues (Java driver) to 17,894 (Lsstcorp Data Management); typical story point is 1 in some projects and 5–8 in others.

Part 4 — The code, file by file

Run order and purpose (all files are in the repository):

#	File	What it does	Input → Output
1	`scripts/download_tawos.py`	Downloads TAWOS (parallel, checks MD5)	internet → `data/raw/TAWOS.sql.zip`
2	`src/extract_tawos.py`	Reads the SQL dump without MySQL	zip → `data/interim/*.parquet / *.csv`
3	`src/preprocess.py`	Filters issues, cleans text, splits sentences	→ `data/processed/issues.parquet`
4	`src/export_splits.py`	Saves the exact train/test split of each pair	→ `results/splits/pair_XX.csv`
5	`src/embed_sbert.py`	SBERT: sentences → 24 × 768 matrices (run once, ~6 min)	→ `data/embeddings/<PROJECT>.npz`
6	`src/splits.py`	Functions for the 80:20 split and the 5 folds	(used by step 9)
7	`src/model.py`	The 3-layer CNN	(used by step 8)
8	`src/train.py`	Training loop: L1 loss, AdamW, early stopping	(used by step 9)
9	`src/run_experiments.py`	Runs all 39 pairs: 5-fold CV, final models, test, baselines (~17 min)	→ `results/main/`
10	`src/analyze_results.py`	Tables, statistics, figures	→ `results/main/tables`, `figures`
11	`scripts/build_report.py`	Builds the report PDF from the results	→ `report/PS8_Group1_Report.pdf`
–	`demo.py`	Predicts story points for any text	—
–	`configs/config.yaml`	**All settings** (model, split, seeds, learning rates)	—

#	File	What it does	Input → Output
–	`configs/pairs.csv`	The 39 pairs from the guidelines	—

What happens inside `run_experiments.py` for one pair (e.g. XD → MESOS):

1. Take MESOS, sort by date, keep the latest 20 % as the test set (527 issues).
2. Take XD (2,408 issues), sort by date, cut into 5 folds.
3. For each of the 4 settings (learning rate × dropout): train 5 times (5-fold) → average validation MAE.
4. Pick the best setting; the number of epochs = median of the best epochs found in the 5 folds.
5. Train on all of XD with seeds 42, 43, 44 → predict the MESOS test issues → MAE / MdAE.
6. Compute the median and mean baselines on the same test issues.
7. Save everything (`pair_results.json`, `cv_folds.csv`, `test_predictions.csv`).

One-liners for each core file (for the viva):

- `embed_sbert.py` — *"Uses the pre-trained SBERT model to turn each sentence into a 768-d vector and stores a 24 × 768 matrix per issue, so training does not have to re-run SBERT."*
- `model.py` — *"Three 1-D convolution layers over the sentence axis, max + mean pooling over real sentences, then two dense layers that output one story point value."*
- `splits.py` — *"Sorts by creation date; target's latest 20 % is the test set; source is cut into five consecutive folds."*
- `train.py` — *"Trains with L1 loss and AdamW; stops early when validation MAE stops improving."*

Part 5 — Numbers to remember

What	Number
Issues after cleaning / projects / pairs	51,777 / 39 / 39
Max sentences per issue / vector size	24 / 768
SBERT model	all-mpnet-base-v2 (frozen)
CNN parameters	~0.72 million
Settings tried per pair	4 (lr 0.001 / 0.0003 × dropout 0.2 / 0.5)
Validation MAE (mean ± std over pairs)	**1.82 ± 1.52** (MdAE 1.19)
Median baseline on validation	2.00 → model better in **34 / 39** pairs (~7 %)
Test MAE / MdAE — our model	**2.21 / 1.85**
Test MAE / MdAE — median baseline	2.16 / 1.88
Test MAE / MdAE — mean baseline	2.56 / 2.19
Model better than median baseline (test)	MAE: 15 of 39 pairs · MdAE: 25 of 39 pairs
Wilcoxon p (MAE, model vs median)	0.59 → not significant
Scale gap vs error correlation	r = 0.76
Seed variation of test MAE	≈ 0.05 points

What	Number
Best pair	C++ driver (MongoDB) → Java driver, MAE 0.29 (almost all issues are 1 point)
Worst pair	Software Cloud → Appcelerator Daemon, MAE 7.18 (source typical 3, target typical 8)
Forward-chaining validation	1.76 (vs 1.82)
Strict chronology (only older source issues)	possible for 29 pairs; model 2.42 = median baseline 2.42
Runtimes	extract 1.5 min · embed ~6 min · experiments ~17 min · GPU RTX 4080

How to read the main figures in the report:

- *Figure 3 (validation folds):* one column per pair; light dots = 5 folds, dark dot = average. Pair 11 (Moodle) varies a lot because Moodle has some very large story points.
- *Figure 4 (test MAE per pair):* blue = our model, orange = median baseline; where only blue is visible they are almost equal. Errors differ more *between pairs* than *between methods*.
- *Figure 5 (training project vs new project):* our model wins on validation (1.82 vs 2.00) but not on test (2.21 vs 2.16) — the key finding.

Part 6 — Discussion (the three things the guidelines ask)

1. Which representation achieves the lowest MAE and MdAE? We only evaluated SBERT (our assignment). **Lowest MdAE: SBERT + CNN (1.85).** Lowest MAE averaged over pairs: the median baseline (2.16 vs 2.21) — tiny, not significant. If all 10,354 test issues are pooled, SBERT + CNN has the lowest MAE too (3.00 vs 3.11). BERT and combined embeddings are other groups' problem statements; we saved our exact splits so they can compare on the same issues.

2. Possible explanations

1. **Each team has its own story point scale** (r = 0.76 between scale gap and error).
2. **Text says only a little about effort**; with a weak signal, the best MAE prediction is close to the median — in about half of the pairs the model predicts almost the same value for every issue.
3. **SBERT captures meaning, not effort** (two similar-sounding issues can need very different work), and it was frozen.
4. **Projects change over time**; the newest issues are the hardest to predict.

3. Limitations / threats to validity

- Story points are subjective → comparing MAE across projects is not completely fair.
- Only 4 hyper-parameter settings; SBERT not fine-tuned.
- Some test sets are tiny (as few as 21 issues) → noisy.
- 5-fold validation trains on some newer issues (checked with forward-chaining: no real effect).
- Our cleaning rules and the Aptana Studio mapping are our own decisions.
- Only open-source, English, one source project per target.

Part 7 — Likely evaluation questions (with short answers)

About the problem

1. **What is your problem statement?** — Predict story points of the latest issues of a project using a model trained on a *different* project, with SBERT sentence embeddings and chronological ordering.
2. **What are story points?** — A relative effort estimate agile teams give each issue (1, 2, 3, 5, 8 …). Each team has its own scale.
3. **Why is cross-project estimation useful?** — A new project has no history; it must use a model trained elsewhere.
4. **Why chronological instead of random?** — Random splits let the model learn from future issues (leakage) and give over-optimistic results. In real life we always predict future issues from past data.

About the representation

5. **Word vs sentence embeddings?** — Word: one vector per word (BERT). Sentence: one vector per sentence (SBERT).
6. **Why SBERT / which model?** — Assigned by the problem statement; we used `all-mpnet-base-v2`, one of the most accurate general-purpose sentence-transformers models, 768-d.
7. **Did you train SBERT?** — No, it is frozen and pre-trained; we only use it to create vectors. This keeps the study about the *representation*, is fast, and avoids overfitting small projects.
8. **Did you write your own embedding model?** — No. We load the pre-trained model with the `sentence-transformers` library; our code prepares sentences, calls `model.encode`, and stores the vectors.
9. **How does a CNN use sentence embeddings?** — Each issue is a 24 × 768 matrix (one row per sentence); the CNN slides over 3 consecutive sentences at a time.
10. **Why 24 sentences?** — Covers 97.1 % of issues; the median issue has only 4.
11. **What is padding / the mask?** — Short issues are filled with zero rows; the mask tells the CNN to ignore them.

About training and evaluation

12. **Why L1 loss?** — It *is* the MAE, so we optimise the metric we report; it is also robust to outliers.
13. **How did you split the data?** — Target: latest 20 % = test (80:20 chronological). Source: 5 time-ordered folds for validation. The earlier 80 % of the target is unused, so nothing is learned from the target.
14. **How did you choose hyper-parameters?** — 4 settings, chosen by average 5-fold validation MAE on the source only. The test set was used once at the end.
15. **Doesn't 5-fold validation use future data?** — In some folds, yes; we checked with forward-chaining validation (always validate on newer data): 1.76 vs 1.82, same conclusion. The test set is always strictly the newest issues.
16. **Why three seeds?** — Neural networks vary with random initialisation; we averaged 3 runs. The variation was only ~0.05 points.
17. **MAE vs MdAE?** — MAE = average error; MdAE = median error, less affected by a few huge mistakes.
18. **What is the median baseline?** — Always predict the training project's median story point. Because the median minimises absolute error, it is a strong baseline for MAE.

About the results

19. **What are your results?** — Test MAE 2.21, MdAE 1.85. Median baseline 2.16 / 1.88. Validation MAE 1.82 ± 1.52.

20. **So your model is not better than the baseline — did it fail?** — It is honest evidence: on the training project it *does* beat the baseline (34/39 pairs), so it learns from text; but that does not transfer to a new project because of different story point scales. This matches published research (Tawosi et al., 2023). It also has the lowest MdAE.
21. **Is the difference significant?** — No; Wilcoxon signed-rank test p = 0.59.
22. **Why does it fail on new projects?** — Different scales (r = 0.76), weak text signal, SBERT measures meaning not effort, and estimation habits change over time.
23. **Best and worst pair?** — Best: C++ driver (MongoDB) → Java driver, MAE 0.29 (almost all 1-point issues). Worst: Software Cloud → Appcelerator Daemon, MAE 7.18 (source typical 3 points, target typical 8).
24. **Which representation is best?** — For our assignment (SBERT): lowest MdAE with SBERT + CNN; MAE equal to the median baseline. BERT and combined are other groups' assignments.
25. **What is strict chronology?** — Training only on source issues created before the target's test issues. Possible for only 29 of 39 pairs (in 10 pairs the source project is newer). Same conclusion: 2.42 vs 2.42.

About data and implementation

26. **How did you clean the data?** — Kept completed issues, 0 < SP ≤ 100 (found values up to 26 million), removed duplicates: 65,427 → 51,777 issues.
27. **What about "C++ driver (Appcelerator)"?** — Not in TAWOS; the only unmatched project is Aptana Studio from the Appcelerator organisation, so we used it and documented the assumption.
28. **How is it reproducible?** — All settings in `configs/config.yaml`, fixed seeds, saved splits, one script (`run_all.ps1`) reruns everything (~40 min on a GPU).
29. **How long does it take / what hardware?** — Embedding ~6 min, all experiments ~17 min on an RTX 4080 laptop GPU.
30. **How would you improve it?** — Use a few labelled target issues to adjust the scale; train on several source projects; fine-tune SBERT on effort data; combine with BERT word embeddings.

Part 8 — Presentation plan (15-minute slot) and demo

Suggested split (≈ 2 minutes each, then questions — swap roles as you like):

Speaker	Topic	Show
Amal Faris M	Problem statement, story points, why cross-project + chronological	Report §1–2
Richie James	Dataset (TAWOS), cleaning steps and numbers	Report §3
Vedavyas S Poika	Method: SBERT sentence vectors → 24 × 768 matrix → 3-layer CNN	Report Fig. 1, Table 1
Muhammed Ismayil	Data split, 5-fold validation, training settings	Report Fig. 2, Table 2
Janardhan Abhiram Bojja	Results, discussion, limitations + live demo	Report Tables 3–4, Figs. 4–5

Live demo (run on the laptop where the project was built — it has the trained models and data):

```
cd C:\Users\abhir\PS8-SBERT-StoryPoint-Estimation
$env:HF_HUB_OFFLINE = "1"          # use the cached SBERT model, no internet needed
.\.venv\Scripts\python.exe demo.py --tag demo --pair 1 --title "Fix the login failure" `
    --description "The login page throws an exception when a user enters an incorrect password."
.\.venv\Scripts\python.exe demo.py --tag demo --pair 1 --show-test 6
```

What to say: *"This model was trained on the XD project only. Here it reads a new issue, splits it into sentences, encodes them with SBERT and the CNN predicts about 2 story points. The second command shows real held-out issues from MESOS with true and predicted values."*

Part 9 — Checklist for the morning of the evaluation

- ☐ Laptop charged; open the report PDF and the GitHub page (https://github.com/abhi230205/PS8-SBERT-StoryPoint-Estimation).
- ☐ Run the demo once (commands above) to warm it up.
- ☐ Everyone can say the main numbers: **2.21 / 1.85 vs 2.16 / 1.88**, validation **1.82**, **34/39**, **p = 0.59**.
- ☐ Everyone can explain: SBERT, the 24 × 768 matrix, the 3-layer CNN, the 80:20 chronological split, 5-fold validation, MAE vs MdAE, why the median baseline is strong, why cross-project is hard.
- ☐ Know the one assumption (Aptana Studio) and the main limitations.

Glossary

Term	Short meaning
Agile / issue / story point	Development style / a task / relative effort estimate
Embedding	Text turned into a vector of numbers
SBERT	BERT trained to give similar vectors to sentences with similar meaning
Frozen model	Pre-trained model used as-is, weights not updated
CNN, kernel, filter	Network that slides small pattern detectors over the input
Pooling	Summarising many positions into one value (max or average)
Dropout	Randomly disabling neurons during training to avoid overfitting
Early stopping	Stop training when validation stops improving
Cross-project	Train on one project, test on another
Chronological split	Train on older issues, test on newer ones
k-fold validation	Split training data into k parts; each part is validation once
Baseline	Simple reference predictor to compare against
MAE / MdAE	Mean / median of absolute errors (lower is better)
Wilcoxon test, p-value	Paired significance test; $p < 0.05$ means a significant difference
Correlation r	Strength of a linear relationship, from −1 to +1
Data leakage	Information from the test period accidentally used in training